

Pemrograman Berorientasi object

Class dan Objek
Ridwan Rismanto

What will we learn

- Dasar membuat class
 - Membuat atribut dan method (behaviour)
 - Dasar membuat object
 - Memanggil method
 - Manipulasi atribut
- 

Contoh kasus: Sistem informasi perbankan

- Kita akan membuat sebuah program sistem informasi perbankan
- Didalam sistem informasi perbankan, salah satunya terdapat data Nasabah.
- Bagaimana cara me-manage data nasabah?
 - Kita buat class Nasabah
- Class Nasabah ini memiliki fitur-fitur sebagai berikut:
 - Memiliki atribut nama nasabah dan saldo
 - Dapat menampilkan jumlah saldo

Cara Mendeklarasikan Class

- Setiap class yang kita buat harus kita simpan di file tersendiri
- Class yang dideklarasikan dengan keyword `public` harus disimpan dengan nama file yang sama, dengan ekstensi `.java`
- Misal, `public class Nasabah`, maka harus disimpan di file **Nasabah.java**
 - INGAT! Huruf besar kecil menentukan

Atribut

- Atribut adalah variabel yang dideklarasikan didalam class
 - Atribut digunakan untuk menyimpan state atau ciri-ciri dari sebuah class
 - Misal, dalam contoh kita kali ini, class Nasabah memiliki atribut **nama** dan **saldo** untuk menyimpan nama dan saldo dari class tersebut
- 

Method

- Method disebut juga function
 - Method adalah behaviour atau perilaku atau “sesuatu yang dapat dilakukan” oleh class tersebut
 - Seperti halnya fungsi, method juga bisa menerima parameter, dan juga bisa memiliki nilai balik atau return value
 - Misal pada contoh kali ini, class Nasabah memiliki method tampilkanSaldo yang dapat menampilkan berapa jumlah saldo yang dimiliki nasabah tersebut
- 

Cara Membuat Object

- Class yang sudah kita buat tidak akan berguna jika kita tidak membuat instance dari class tersebut
 - Atau dengan kata lain, membuat object dari class tersebut
- Cara membuat object adalah dengan menggunakan keyword **new**
- Misal, membuat object bernama **nsb** dari class **Nasabah**:

```
Nasabah nb = new Nasabah ( ) ;
```



Nama class

Nama objek

Nama class lagi

Class Nasabah

- Pertama-tama kita buat class Nasabah
- Class Nasabah ini memiliki atribut nama nasabah dan saldo, dan method tampilkanSaldo

```
public class Nasabah
{
    public String nama;
    public int saldo;

    public void tampilkanSaldo()
    {
        System.out.println("Saldo anda adalah " + saldo);
    }
}
```

Nama class

Atribut nama
dan saldo

Method/function
tampilkanSaldo

Didalam class Nasabah

- Amati kembali class Nasabah
 - Keyword **public** adalah access modifier
 - Keyword **void** menandakan bahwa method/function tersebut tidak mengembalikan nilai
- 

Test unit

- Jika diperhatikan, class Nasabah tidak memiliki method main().
- Sehingga jika kita mencoba mengeksekusi class tersebut dengan perintah `java Nasabah`, maka akan error.
- Memang sengaja dibuat seperti itu, karena kita akan memisahkan method `main()` ini ke class yang lain, yang kita sebut dengan Test unit.
- Test unit kita gunakan sebagai program untuk menguji class-class yang sudah kita buat sebelumnya.
- Untuk test unit kita kali ini, kita akan membuat class **TestPerbankan** sebagai berikut:

Class TestPerbankan

```
public class TestPerbankan
{
    public static void main(String[] args)
    {
        Nasabah nsb = new Nasabah();

        nsb.nama = "Bill Gates";
        nsb.saldo = 1000;

        nsb.tampilkanSaldo();
    }
}
```

Method main

Pembuatan objek **nsb**
dari class Nasabah

Mengisi atribut nama dan
saldo dari objek **nsb**

Memanggil method
tampilkanSaldo dari objek
nsb

```
E:\Java\Perbankan>javac TestPerbankan.java
```

```
E:\Java\Perbankan>java TestPerbankan
Saldo anda adalah 1000
```

Didalam class TestNasabah

- Amati kembali class TestPerbankan
- Class ini adalah sebuah aplikasi karena memiliki method `main()` yang akan di-eksekusi pertama kali oleh JVM
- Method `main()` adalah method yang istimewa karena dia adalah **static**
 - Method static membolehkan kita memanggil method tersebut tanpa harus menciptakan object dari class tersebut
- Didalam method `main()`, kita membuat object **nsb** dari class **Nasabah**
- Kemudian kita mengisi atribut **nama** dan **saldo** dari objek **nsb**
- Kemudian kita memanggil method **tampilkanSaldo()** dari object **nsb**
- Method pada object dapat dipanggil dengan menggunakan **dot separator (.)**
 - Contoh: `nsb.tampilkanSaldo()`

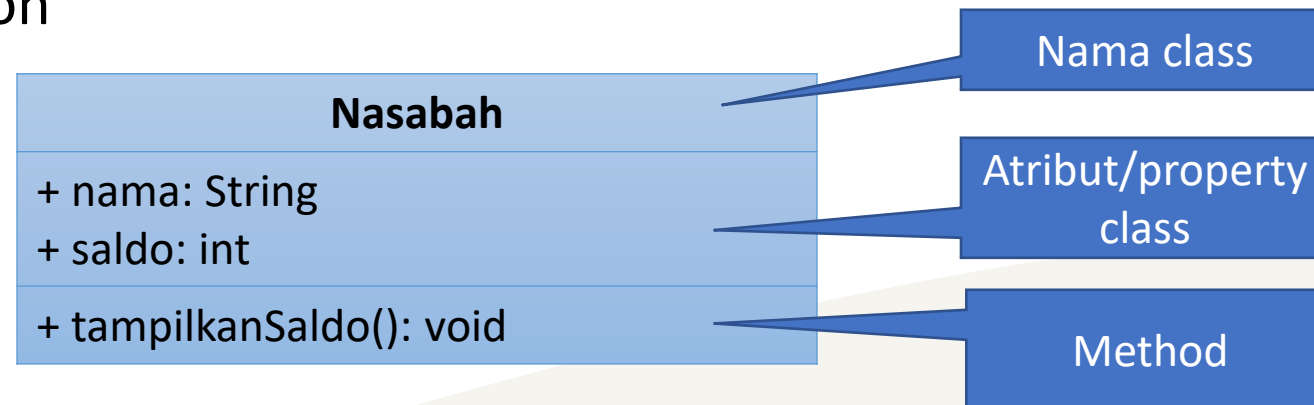
Melakukan compile beberapa class

- Pada saat meng-compile, kita hanya perlu meng-compile class yang ada method `main()` nya saja, selanjutnya JVM akan otomatis mencari class-class lain yang dibutuhkan didalam direktori/folder yang sama.
- Dalam contoh kita ini, terdapat class **Nasabah** dan **TestPerbankan**, kita cukup mengompile class `TestPerbankan` saja.
- Class **Nasabah** akan otomatis di-compile oleh JVM.

Class Diagram

- Class diagram adalah sebuah diagram yang digunakan untuk menggambarkan skema class pada sistem yang berorientasi object
- Didalam class diagram terdapat:
 - Nama class
 - Atribut/property/variabel class
 - Method/function

Contoh:



Class diagram untuk Nasabah

Nasabah
+ nama: String + saldo: int
+ tampilkanSaldo(): void

Method yang memiliki parameter

- Berikutnya kita modifikasi class Nasabah, yaitu pada method tampilkanSaldo kita ubah agar menerima parameter berupa kode mata uang.

```
public class Nasabah
{
    public String nama;
    public int saldo;

    public void tampilkanSaldo(String mataUang)
    {
        System.out.println("Saldo anda adalah " + mataUang + saldo);
    }
}
```

Menerima parameter **mataUang** dengan tipe **String**

Memanggil parameter **mataUang** tersebut agar muncul ke layar

TestPerbankan

- Berikutnya kita compile dan jalankan lagi class TestPerbankan apakah perubahan yang kita lakukan pada class Nasabah tadi bekerja sesuai yang kita harapkan
 - Kita masukkan string "IDR" sebagai parameter pada method tampilkanSaldo

```
public class TestPerbankan
{
    public static void main(String[] args)
    {
        Nasabah nsb = new Nasabah();
        nsb.nama = "Bill Gates";
        nsb.saldo = 1000;

        nsb.tampilkanSaldo("IDR");
    }
}
```

Masukkan string
"IDR" sebagai
parameter

```
E:\Java\Perbankan>javac TestPerbankan.java
```

```
E:\Java\Perbankan>java TestPerbankan
Saldo anda adalah IDR1000
```

Class diagram Nasabah - updated

- Karena tadi kita sudah menambahkan parameter **mataUang** pada method **tampilkanSaldo**, maka sekarang class diagram Nasabah menjadi seperti ini:

Nasabah
+ nama: String + saldo: int
+ tampilkanSaldo(mataUang: String): void

Membuat lebih dari satu object

- Kita dapat membuat lebih dari satu object dari class yang sama
- Kita coba membuat dua objek nasabah, yaitu nsb1 dan nsb2, pada class TestPerbankan

```
public class TestPerbankan
{
    public static void main(String[] args)
    {
        Nasabah nsb1 = new Nasabah();
        Nasabah nsb2 = new Nasabah();

        nsb1.nama = "Bill Gates";
        nsb1.saldo = 1000;

        nsb2.nama = "Ridwan Rismanto";
        nsb2.saldo = 9999;

        nsb1.tampilkanSaldo("IDR");
        nsb2.tampilkanSaldo("IDR");
    }
}
```

```
E:\Java\Perbankan>java TestPerbankan
Saldo anda adalah IDR1000
Saldo anda adalah IDR9999
```

Modifikasi Class

- Ketika kita memodifikasi class, maka perubahannya akan langsung ber-efek ke semua objek yang dibuat dari class tersebut
- Misal kita ubah method **tampilkanSaldo** pada class Nasabah agar menampilkan nama nasabah

```
public class Nasabah
{
    public String nama;
    public int saldo;

    public void tampilkanSaldo(String mataUang)
    {
        System.out.println("Saldo " + nama + " adalah " + mataUang + saldo);
    }
}
```

Menampilkan
atribut **nama**

```
E:\Java\Perbankan>java TestPerbankan
Saldo Bill Gates adalah IDR1000
Saldo Ridwan Rismanto adalah IDR9999
```

Access Modifier

- Kita tadi mendeklarasikan atribut **nama**, **saldo** dan method **tampilkanSaldo()** dengan access modifier **public**, agar dapat diakses dari class lain (yaitu class TestPerbankan)
- Beberapa akses modifier yang sering dipakai:
 - Private
 - Hanya bisa diakses oleh method yang ada didalam class yang sama
 - Public
 - Dapat diakses oleh method yang ada diluar class
 - Protected
 - Hanya dapat diakses oleh class turunannya (child class)

Rangkuman class Nasabah

- Dari percobaan kita barusan, dapat kita simpulkan bahwa:
 - Class Nasabah memiliki atribut nama dan saldo. Class ini juga memiliki method untuk menampilkan nama dan saldo nasabah, dan juga menerima parameter mata uang.
- 

Question?